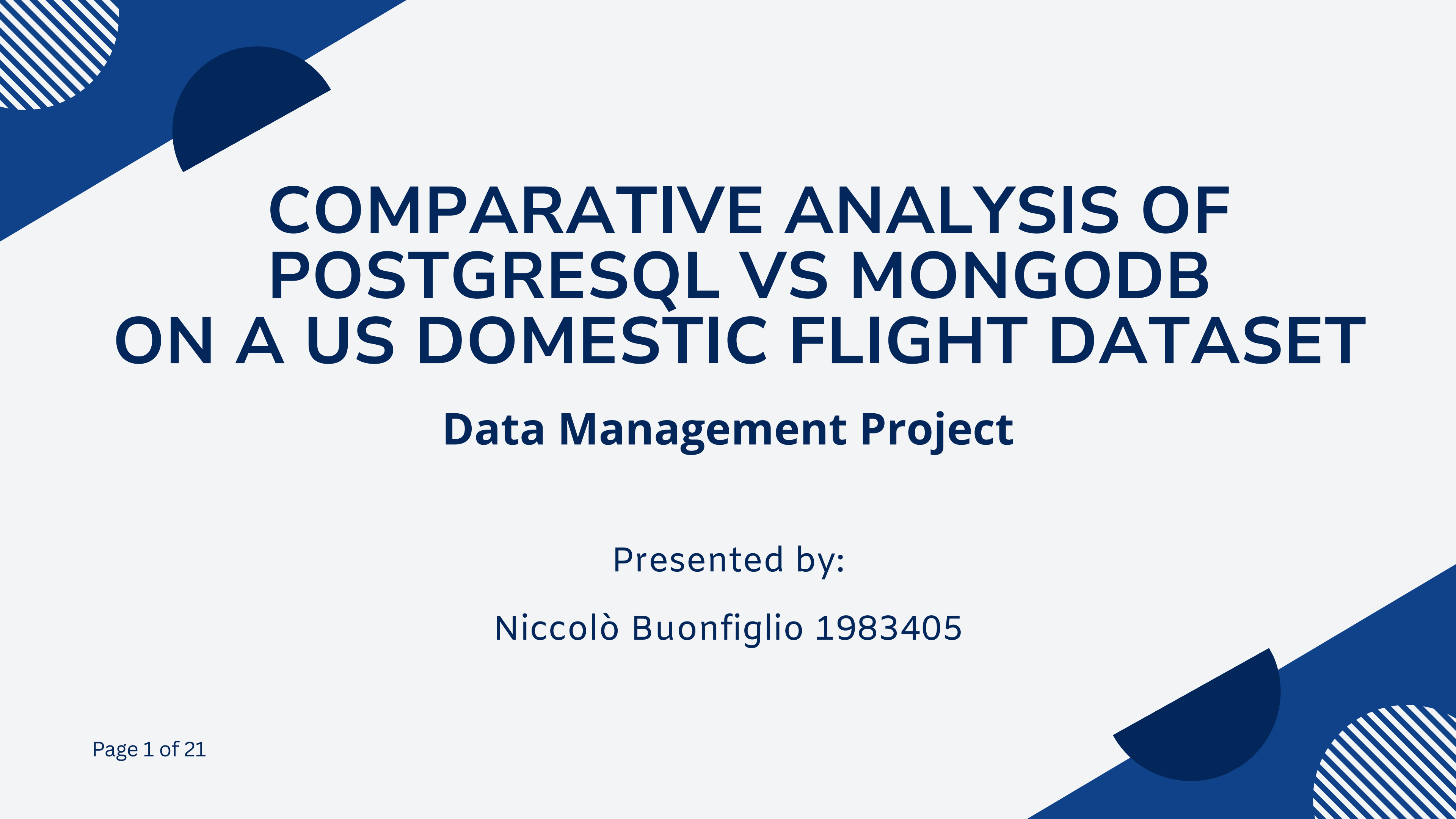




COMPARATIVE ANALYSIS OF POSTGRESQL VS MONGODB ON A US DOMESTIC FLIGHT DATASET

Data Management Project

Presented by:

Niccolò Buonfiglio 1983405



STARTING POINT

US domestic flights csv dataset obtained from Kaggle, international airport and airline csv datasets obtained from OpenFlights and list of airline codes from Wikipedia. Un-normalized data, duplicates, wrong data formats. Large time window and data incompleteness.

GOAL

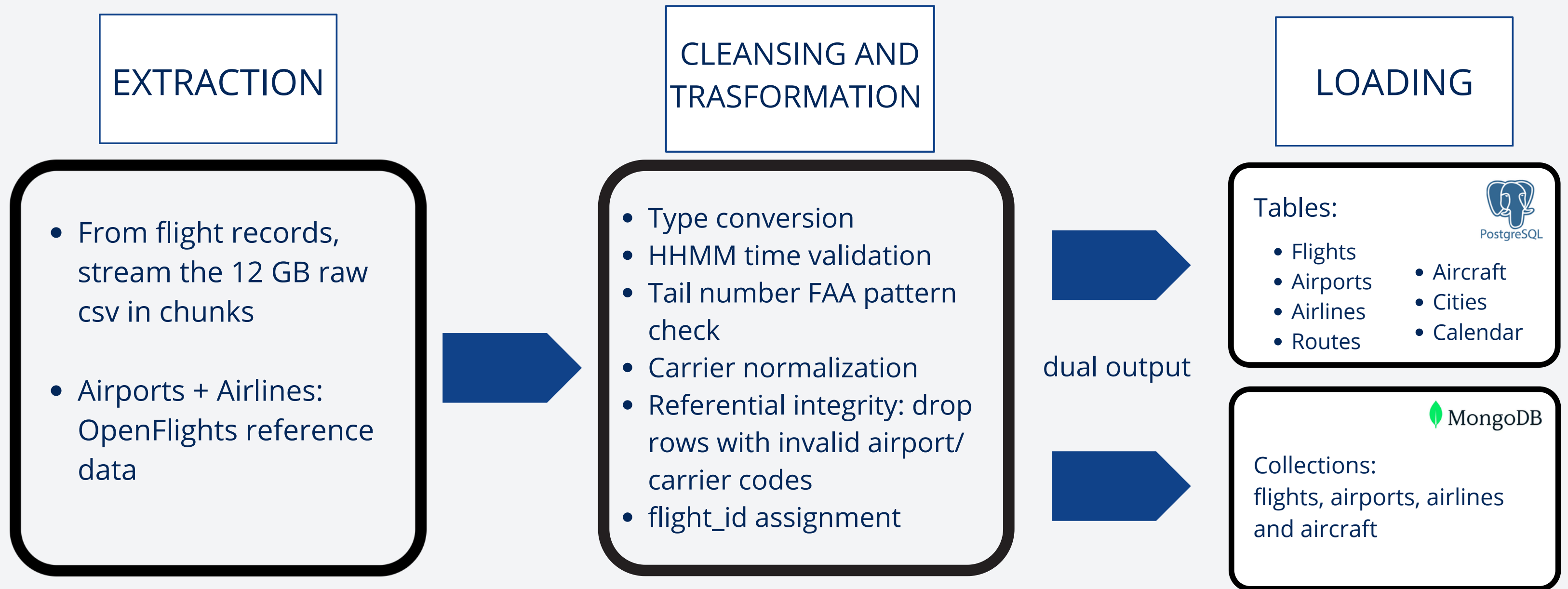
Compare database performance across relational and document models using PostgreSQL and MongoDB.

DESIGN

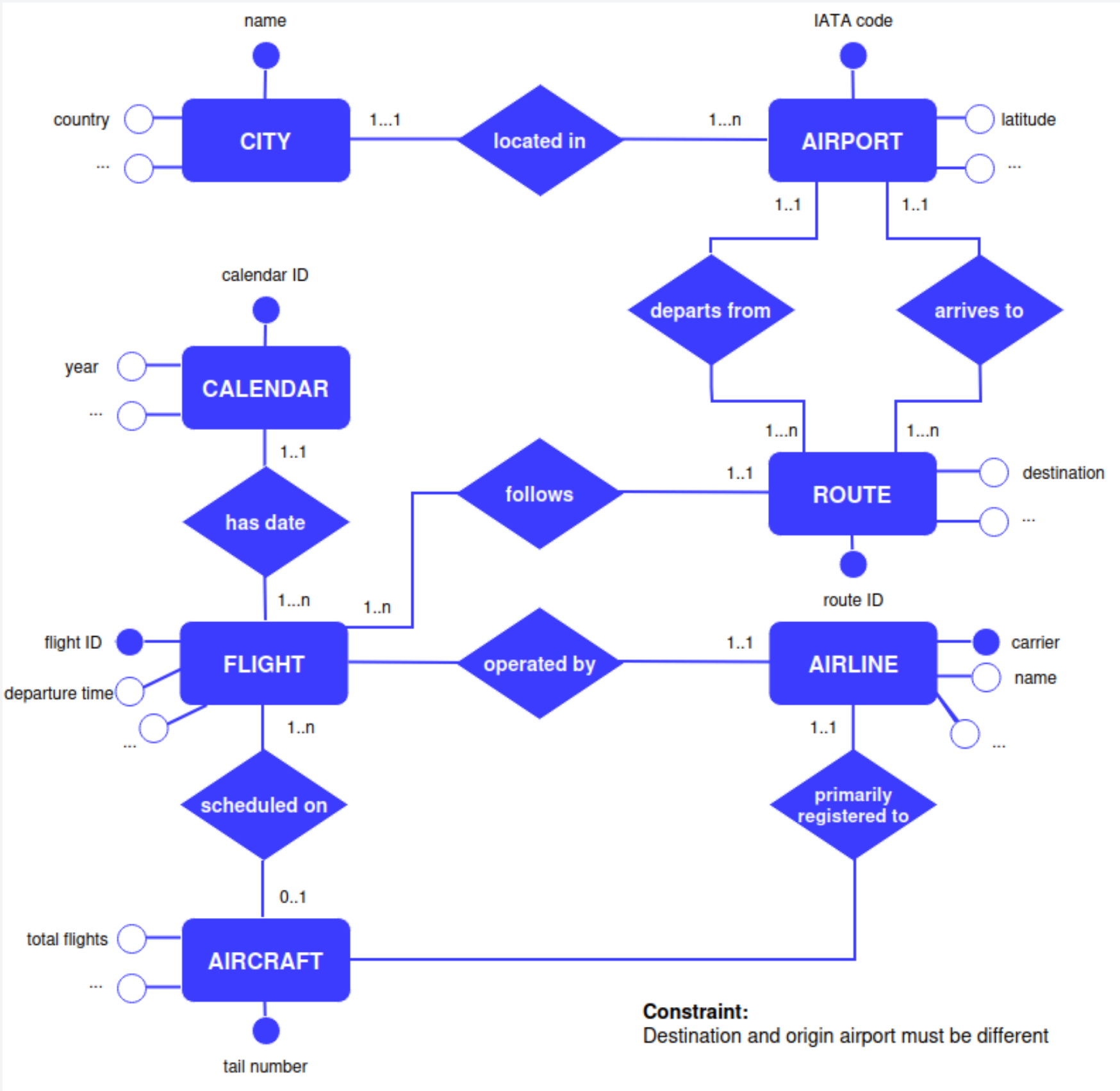
Comparative database architecture for static data which is unsuitable for live updates due to its historical nature. The dataset is useful for historical comparisons.

ETL PIPELINE OVERVIEW

Preprocessing script implements a streaming ETL pipeline that transforms flight records into dual-format outputs for both PostgreSQL and MongoDB.



ER SCHEMA



Entity	Attributes	Identifier
Flight	Arr_delay, dep_delay, carrier, ...	Flight id
Airport	Name, city, latitude,	Airport IATA code
Airline	Name, alias, ICAO, country, ...	Airline carrier number
Aircraft	Primary airline, total flights, ...	Aircraft tail number
City	Country, timezone, dst, ...	City name
Calendar	Day, month, year, ...	Calendar id
Route	Origin airport, destination airport, ...	Route id

Relationship	Description	Entities involved
Departs from	It associates an origin airport to a route	Route (1,n), Airport (1,1)
Arrives to	It associates a destination airport to a route	Route (1,n), Airport (1,1)
Operated by	It associates a flight to the airline of the plane	Flight (0,n), Airline (1,1)
Located in	It associates an airport to the city it's located in	Airport (1,n), City (1,1)
Scheduled on	It associates a flight to the tail number of the aircraft	Flight (1,n), Aircraft (0,1)
Follows	It associates a flight to the route that it follows	Flight (1,n), Route (1,1)
Has date	It associates a flight to a calendar date	Flight (1,n), Calendar (1,1)
Primarily registered to	It associates an aircraft to its primary airline	Aircraft (0,n), Airline (1,1)

POSTGRESQL: RELATIONAL SCHEMA



Following the ER schema in the previous slide

Foreign keys:

airports(city_name) REFERENCES cities(city_name)
routes(origin) NOT NULL REFERENCES airports(iata_code)
routes(dest) NOT NULL REFERENCES airports(iata_code)
aircraft(primary_airline) REFERENCES airlines(unique_carrier)

flights(tail_num) REFERENCES aircraft(tail_number)
flights(unique_carrier) NOT NULL REFERENCES airlines(unique_carrier)
flights(route_id) NOT NULL REFERENCES routes(routes_id)
flights(calendar_id) NOT NULL REFERENCES calendar(calendar_id)

Index strategy:

Partial indexes on columns reduce index size, especially speeding up "delayed flights" queries.
Foreign key indexes prevent full table scans during JOINS.

Flights: [unique_carrier], [route_id], [calendar_id], [tail_num], [arr_delay], [dep_delay], [(cancelled, calendar_id)]

Airports: [city_name], [(latitude, longitude)]

Routes: [origin], [dest]

Calendar: [year], [(year, month)]

Aircraft: [primary_airline]

City: [country]

MONGODB: DOCUMENT MODEL

MongoDB was structured in 4 collections: **Flights, Airports, Airlines, Aircraft**

- Airports collection holds information about the geography, with info naturally nested via embedded city objects.
- Flight attributes, delay causes, temporal factors and airport info were nested inside of the flights collection: this is the core
- Airlines and aircraft collections are vital for data completeness and independent lookups
- Some IDs are referenced across documents (e.g. flights contains an aircraft tail number). This must be maintained when accessing the database (for application-level consistency or manual lookup when needed).

Each flight document contains multiple nested subdocuments and uses GeoJSON data.

Indexes:

Flights: [flight_id, unique], [tail_num], [(year, month)], [(origin.location, GEOSPHERE)], [(dest.location, GEOSPHERE)], [(cancelled, origin.iata)], [delays.arr_delay], [delays.dep_delay], [carrier.code], [(origin.iata, dest.iata, year)]

Airports: [iata_code, unique], [city_name], [country], [(location, GEOSPHERE)]

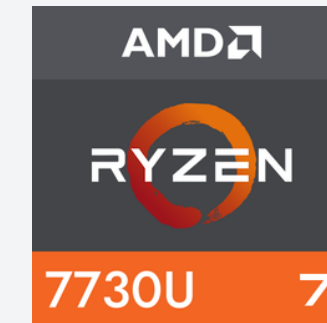
Airlines: [unique_carrier, unique]

Aircrafts: [tail_num, unique], [primary_airline]

TESTING DETAILS

Hardware components:

- Model: Dell 15 Laptop (DC15255)
- Processor: AMD Ryzen 7 7730U
- Memory: 16GB DDR4 RAM
- OS: Linux-mint



Testing phase only includes ~2% of total flights, due to hardware constraints. Each test analysis run per query will include 12 total executions per query, where the results of the first 2 runs of each test were discarded to account for caching.

QUERY ANALYSIS

Tests were ran for the following queries:

- **Mass insertion:** insertion of 1000 new flight records
- **Mass deletion:** deletion of 1000 flight records
- **Aggregation without JOIN:** counting flights by carriers and arrival time
- **Heavy aggregation with JOIN:** grouping mid-week flights by route-month combinations that had more than 20 flights during weekdays in 2007-2008, avg. arrival delay and severe delays.
- **Embedded field access:** filtering flights from a specific year departing from five major cities by four specific airlines and distance range, counting number of flights and calculating average delay.
- **Range nested query:** finding non-cancelled flights with a departure delay that ranges from 60 to 90 minutes.
- **Embedded field increment:** updating the aircraft's total flights count and the flight's carrier delay of not cancelled flights with large arrival and carrier delay.
- **Conditional aggregation:** airline-city routes by filtering high-traffic pairs, counting distinct aircraft per route and measuring average distance.
- **Multi-table JOIN without aggregation:** severe flight delays during the peak winter travel months, comparing performance across airlines and including city details.
- **Self-lookup + aggregation:** aircrafts that operated the same route multiple times on a single day, aggregating flight frequency and average delay.

TEST RESULTS

Mass insertion

Insertion of 1000 new flight records

Average time: 72.76 ms Min time: 68.39 ms Max time: 76.13 ms Std. derivation: ± 3.48 ms Excluding first 2 runs, using 10 executions	Average time: 40.88 ms Min time: 37.02 ms Max time: 42.43 ms Std. derivation: ± 2.47 ms Excluding first 2 runs, using 10 executions
---	---

**MongoDB
wins by 1.78x**

- MongoDB is clearly faster.
- Document databases are faster for batch operations because they skip transaction overhead and constraint checking that relational databases need.
- PostgreSQL, due to foreign keys and normalized table structures, introduces additional write overhead per operation.

PostgreSQL query

```
INSERT INTO flights (  
    flight_id, tail_num, unique_carrier, route_id, calendar_id, flight_num,  
    cancelled, diverted, cancellation_code,  
    crs_dep_time, dep_time, crs_arr_time, arr_time,  
    crs_elapsed_time, actual_elapsed_time, air_time, taxi_in, taxi_out,  
    arr_delay, dep_delay, carrier_delay, weather_delay, nas_delay,  
    security_delay, late_aircraft_delay  
)
```

MongoDB query

```
db.flights.insert_many(  
    { flight_id: "...", year: "...", ...,  
      carrier: { code: "...", name: "...", ... },  
      origin: { iata: "...", location: { type: "...", coordinates: ["...", "..."] }, [...] },  
      dest: { [...] } },  
      delays: { dep_delay: "...", arr_delay: "...", [...] },  
      ...  
    ], { ordered: False })
```

TEST RESULTS

- MongoDB is even faster.
- PostgreSQL must cascade deletes or maintain referential integrity across related tables
- Due to its structure, MongoDB has the power to perform document deletion with fewer overhead

1000 test records were created beforehand with ID range from x to x+1000, in order to not delete any relevant existing data in the databases.

PostgreSQL query

```
DELETE FROM flights
WHERE flight_id BETWEEN run_start_id AND run_end_id;
```

Mass deletion

Deletion of 1000 flight records

Average time:
48.02 ms

Min time: 44.66 ms
Max time: 52.19 ms
Std. derivation: ± 3.42 ms

Excluding first 2 runs, using 10 executions

Average time:
23.64 ms

Min time: 21.75 ms
Max time: 25.32 ms
Std. derivation: ± 1.52 ms

Excluding first 2 runs, using 10 executions

**MongoDB
wins by 2.03x**

MongoDB query

```
db.flights.deleteMany({
  flight_id: {
    $gte: NumberLong(runStartId),
    $lte: NumberLong(runEndId)
  }
});
```

TEST RESULTS

Aggregation without JOIN

Counting flights by carrier and arrival time

PostgreSQL query

```
SELECT f.unique_carrier, f.arr_time, COUNT(*) as
count
FROM flights f
WHERE f.arr_time IS NOT NULL
GROUP BY f.unique_carrier, f.arr_time
ORDER BY count DESC
```

MongoDB query

```
db.flights.aggregate([
  { $match: {"actual.arr_time": { $ne: null } } },
  { $group: { _id: { carrier: "$carrier.code",
arr_time: "$actual.arr_time" }, count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
```

- PostgreSQL is faster: SQL's native GROUP BY operations are ideal
- MongoDB's aggregation adds heavy processing overhead

Interesting result for data analysis:
Southwest Airlines (WN) has high-frequency regular schedules at :00, :30, :50, ...
Other airlines have fewer flights sharing the exact arrival minute time.

Average time:
433.14 ms

Min time: 421.03 ms
Max time: 460.46 ms
Std. derivation: ± 13.49 ms

Excluding first 2 runs, using 10 executions

Average time:
1484.10 ms

Min time: 1439.45 ms
Max time: 1507.77 ms
Std. derivation: ± 19.76 ms

Excluding first 2 runs, using 10 executions

**PostgreSQL
wins by 3.42x**

Sample result

Carrier: WN, Arrival Time: 1650, Count: 659
Carrier: WN, Arrival Time: 1630, Count: 657
Carrier: WN, Arrival Time: 2100, Count: 644
Carrier: WN, Arrival Time: 1700, Count: 642
Carrier: WN, Arrival Time: 1900, Count: 635
[...]

TEST RESULTS

Heavy aggregation with JOIN

Grouping by route-month combinations that had more than 20 flights during weekdays in 2007-2008, avg. arrival delay and severe delays.

PostgreSQL query

```
SELECT r.origin, r.dest, r.distance as route_distance, c.year, c.month, COUNT(*) as flight_count,
ROUND(AVG(f.arr_delay)::numeric, 2) as avg_arr_delay, SUM(CASE WHEN f.arr_delay > 30
THEN 1 ELSE 0 END) as delays_over_30min
FROM flights f
JOIN routes r ON f.route_id = r.route_id
JOIN calendar c ON f.calendar_id = c.calendar_id
JOIN airlines al ON f.unique_carrier = al.unique_carrier
WHERE c.year IN (2007, 2008) AND f.cancelled = false AND c.day_of_week BETWEEN 2 AND 6
GROUP BY r.origin, r.dest, r.distance, c.year, c.month
HAVING COUNT(*) > 20
ORDER BY flight_count DESC;
```

Average time:
180.05 ms

Min time: 176.42 ms
Max time: 184.66 ms
Std. derivation: ± 2.26 ms

Excluding first 2 runs, using 10
executions

Average time:
883.42 ms

Min time: 873.07 ms
Max time: 892.71 ms
Std. derivation: ± 7.32 ms

Excluding first 2 runs, using 10
executions

**PostgreSQL
wins by 4.91x**

MongoDB query

```
db.flights.aggregate([
{ $match: { year: { $in: [2007, 2008] }, cancelled: false, day_of_week: { $in: [2, 3, 4, 5, 6] } } },
{ $group: { _id: { origin: "$origin.iata", dest: "$dest.iata", year: "$year", month: "$month" },
flight_count: { $sum: 1 }, avg_arr_delay: { $avg: "$delays.arr_delay" }, delays_over_30min: {
$sum: { $cond: [{ $gt: ["$delays.arr_delay", 30] }, 1, 0] } }, route_distance: { $first: "$distance" }
} },
{ $match: { flight_count: { $gt: 20 } } },
{ $project: { origin: "$_id.origin", dest: "$_id.dest", year: "$_id.year", month: "$_id.month",
flight_count: 1, avg_arr_delay: { $round: ["$avg_arr_delay", 2] }, delays_over_30min: 1,
route_distance: 1, _id: 0 } },
{ $sort: { flight_count: -1 } }
]);
```

- PostgreSQL dominates: very efficient join execution
- MongoDB's performance, when the query complexity gets heavier, degrades much faster than PostgreSQL's.

TEST RESULTS

Embedded field access

Filters flights from a specific year departing from five major cities by four specific airlines and distance range, counting number of flights and calculating average delay.

PostgreSQL query

```
SELECT c.city_name, al.airline_name, COUNT(*) as flights,
ROUND(AVG(f.arr_delay)::numeric, 2) as avg_delay
FROM flights f
JOIN airlines al ON f.unique_carrier =
al.unique_carrier
JOIN routes r ON f.route_id = r.route_id
JOIN airports a ON r.origin = a.iata_code
JOIN cities c ON a.city_name = c.city_name
JOIN calendar cal ON f.calendar_id = cal.calendar_id
WHERE cal.year = 2008 AND f.cancelled = false
AND c.city_name IN ('Chicago', 'Atlanta', 'Dallas',
'Denver', 'Los Angeles')
AND r.distance BETWEEN 300 AND 1500
AND al.airline_name IN ('Southwest Airlines',
'American Airlines', 'United Airlines', 'Delta Air Lines')
GROUP BY c.city_name, al.airline_name
HAVING COUNT(*) > 100
ORDER BY flights DESC
```

MongoDB query

```
db.flights.aggregate([
  {$match: { year: 2008, cancelled: false,
"origin.city.city_name": { $in: ["Chicago", "Atlanta", "Dallas",
"Denver", "Los Angeles"] }, distance: { $gte: 300, $lte: 1500
},
  "carrier.name": { $in: ["Southwest Airlines",
"American Airlines", "United Airlines", "Delta Airlines"] } } },
  { $group: { _id: { city: "$origin.city.city_name", airline:
"$carrier.name" },
  flight_count: { $sum: 1 }, avg_delay: { $avg:
"$delays.arr_delay" } } },
  { $match: { flight_count: { $gt: 100 } } },
  { $project: { city_name: "$_id.city", airline_name:
"$_id.airline", flight_count: 1, avg_delay: { $round:
["$avg_delay", 2] }, _id: 0 } },
  { $sort: { flight_count: -1 } }
])
```

Average time:
583.84 ms

Min time: 576.19 ms
Max time: 597.08 ms
Std. derivation: ± 5.14 ms

Excluding first 2 runs, using 10
executions

Average time:
215.81 ms

Min time: 214.57 ms
Max time: 217.12 ms
Std. derivation: ± 1.07 ms

Excluding first 2 runs, using 10
executions

**MongoDB
wins by 2.71x**

Sample result

Atlanta - Delta Air Lines: 2,095 flights, Avg Delay: 8.98 min
Denver - United Airlines: 1,269 flights, Avg Delay: 11.47 min
Chicago - United Airlines: 1,172 flights, Avg Delay: 18.35 min
Chicago - Southwest Airlines: 1,002 flights, Avg Delay: 6.91 min
Chicago - American Airlines: 985 flights, Avg Delay: 17.19 min
Los Angeles - Southwest Airlines: 645 flights, Avg Delay: 7.98 min
Denver - Southwest Airlines: 543 flights, Avg Delay: 4.28 min
Dallas - Southwest Airlines: 395 flights, Avg Delay: 6.44 min
Los Angeles - United Airlines: 203 flights, Avg Delay: 7.73 min
Los Angeles - American Airlines: 201 flights, Avg Delay: 10.97 min
[...]

- MongoDB wins with margin: it accessed fields directly without JOINS
- Accessing nested document fields is great in MongoDB, while PostgreSQL must join across multiple normalized tables (in this case there is a total of 5 JOINS), adding query complexity and join costs.

TEST RESULTS

Range nested query

Finds non-cancelled flights with a departure delay that ranges from 60 to 90 minutes.

- PostgreSQL is greatly faster
- Range queries on nested fields suffer from noticeable issues in document-based models.
- PostgreSQL's partial indexes and efficient column projection deliver a great performance.

PostgreSQL query

```
SELECT f.flight_id, f.dep_delay, f.arr_delay, al.airline_name,  
r.origin, r.dest, cal.year, cal.month, cal.day_of_month  
FROM flights f  
JOIN airlines al ON f.unique_carrier = al.unique_carrier  
JOIN routes r ON f.route_id = r.route_id  
JOIN calendar cal ON f.calendar_id = cal.calendar_id  
WHERE f.dep_delay BETWEEN 60 AND 90 AND f.cancelled = false  
ORDER BY f.dep_delay DESC;
```

MongoDB query

```
db.flights.find(  
{ "delays.dep_delay": { $gte: 60, $lte: 90 }, cancelled:  
false },  
{ flight_id: 1, "delays.dep_delay": 1, "delays.arr_delay": 1,  
year: 1, month: 1, day_of_month: 1, "carrier.name": 1,  
"origin.iata": 1, "dest.iata": 1, _id: 0 })  
.sort({ "delays.dep_delay": -1 });
```

Average time:
190.40 ms

Min time: 152.85 ms
Max time: 229.53 ms
Std. derivation: ± 19.02 ms

Excluding first 2 runs, using 10
executions

Average time:
1000.62 ms

Min time: 982.77 ms
Max time: 1033.95 ms
Std. derivation: ± 18.37 ms

Excluding first 2 runs, using 10
executions

**PostgreSQL
wins by 5.26x**

TEST RESULTS

Embedded field increment

Updates the aircraft's total flights count and the flight's carrier delay of not cancelled flights with large arrival and carrier delay.

PostgreSQL query

```
UPDATE aircraft
SET total_flights = total_flights + 1
WHERE tail_num IN (
  SELECT f.tail_num
  FROM flights f
  WHERE f.arr_delay > 180
  AND carrier_delay > 30
  AND f.cancelled = false
  AND f.tail_num IS NOT NULL
);

UPDATE flights
SET carrier_delay = carrier_delay + 5
WHERE arr_delay > 180
  AND carrier_delay > 30
  AND cancelled = false;
```

MongoDB query

```
db.flights.updateMany(
  { "delays.arr_delay": { $gt: 180 }, "delays.carrier_delay": { $ne: null, $gt: 30 },
    "aircraft.total_flights": { $ne: null, $gt: 100 }, "cancelled": false },
  { $inc: {"aircraft.total_flights": 1, "delays.carrier_delay": 5 } }
)
```

**MongoDB
wins by 1.38x**

Average time:
180.97 ms

Min time: 155.43 ms
Max time: 235.21 ms
Std. derivation: ± 23.47 ms

Excluding first 2 runs, using 10
executions

Average time:
131.44 ms

Min time: 124.64 ms
Max time: 140.07 ms
Std. derivation: ± 5.74 ms

Excluding first 2 runs, using 10
executions

- MongoDB is faster
- Unlike PostgreSQL, thanks to its document structure, MongoDB can perform a single operation in a nested document.

Conditional aggregation

Finds flights from 2008 that weren't cancelled, counts how many flights combinations airline-origin city there are, and returns only those with more than 200 flights, showing distinct aircraft count and average distance

PostgreSQL query

```
SELECT al.airline_name, c.city_name, COUNT(*) as
flight_count, COUNT(DISTINCT f.tail_num) as
aircraft_used, ROUND(AVG(r.distance)::numeric, 2)
as avg_distance
FROM flights f
JOIN airlines al ON f.unique_carrier =
al.unique_carrier
JOIN routes r ON f.route_id = r.route_id
JOIN airports a ON r.origin = a.iata_code
JOIN cities c ON a.city_name = c.city_name
JOIN calendar cal ON f.calendar_id = cal.calendar_id
WHERE cal.year = 2008 AND f.cancelled = false
GROUP BY al.airline_name, c.city_name
HAVING COUNT(*) > 200
ORDER BY flight_count DESC
```

MongoDB query

```
db.flights.aggregate([
  { $match: { year: 2008, cancelled: false } },
  { $group: { _id: { airline: "$carrier.name", city:
"$origin.city.city_name"},
    flight_count: { $sum: 1 },
    aircraft_used: { $addToSet: "$tail_num" },
    avg_distance: { $avg: "$distance" }
  } },
  { $addFields: { aircraft_used_count: { $size:
"$aircraft_used" } } },
  { $match: { flight_count: { $gt: 200 } } },
  { $sort: { flight_count: -1 } },
  { $project: { airline_name: "$_id.airline", city_name:
"$_id.city", flight_count: 1, aircraft_used:
"$aircraft_used_count", avg_distance: { $round:
["$avg_distance", 2] }, _id: 0 }
}]
```

Sample result

```
Airline: American Airlines, City: Dallas-Fort Worth, Flights: 3,117, Aircraft: 521, Avg Dist: 915.48 miles
Airline: Delta Air Lines, City: Atlanta, Flights: 2,996, Aircraft: 380, Avg Dist: 772.68 miles
Airline: Atlantic Southeast Airlines, City: Atlanta, Flights: 2,564, Aircraft: 162, Avg Dist: 450.15 miles
Airline: American Eagle Airlines, City: Dallas-Fort Worth, Flights: 1,746, Aircraft: 328, Avg Dist: 455.03 miles
Airline: United Airlines, City: Chicago, Flights: 1,722, Aircraft: 380, Avg Dist: 926.58 miles
[...]
```

TEST RESULTS

Average time: 272.49 ms Min time: 269.51 ms Max time: 275.83 ms Std. derivation: ± 2.11 ms Excluding first 2 runs, using 10 executions	Average time: 359.53 ms Min time: 357.69 ms Max time: 363.22 ms Std. derivation: ±1.78 ms Excluding first 2 runs, using 10 executions
---	--

PostgreSQL
wins by 1.31x

- Interesting result: PostgreSQL wins despite using 5 JOINS, compared to the embedded route data in MongoDB
- PostgreSQL's COUNT(DISTINCT f.tail_num) is optimized to use hash aggregation without storing the entire set per group in the same way
- MongoDB has to use \$addToSet for aircraft, which stores full array per group, causing massive memory overhead

TEST RESULTS

Multi-table JOIN without aggregation

Analyzes severe flight delays during the peak winter travel months (November-December 2007), comparing performance across airlines and including city details.

PostgreSQL query

```
SELECT
  f.flight_id,
  f.flight_num,
  f.arr_delay,
  f.dep_delay,
  al.airline_name,
  COALESCE(a.tail_num, 'NO_AIRCRAFT') as tail_num,
  COALESCE(a.total_flights, 0) as aircraft_total_flights,
  org.city_name as origin_city,
  dest.city_name as dest_city,
  org_city.timezone_offset as origin_tz,
  dest_city.timezone_offset as dest_tz
FROM flights f
INNER JOIN airlines al ON f.unique_carrier = al.unique_carrier
INNER JOIN routes r ON f.route_id = r.route_id
INNER JOIN airports org ON r.origin = org.iata_code
INNER JOIN airports dest ON r.dest = dest.iata_code
LEFT JOIN aircraft a ON f.tail_num = a.tail_num
LEFT JOIN cities org_city ON org.city_name = org_city.city_name
LEFT JOIN cities dest_city ON dest.city_name =
dest_city.city_name
INNER JOIN calendar cal ON f.calendar_id = cal.calendar_id
WHERE f.arr_delay > 60
  AND f.dep_delay > 30
  AND cal.year = 2007
  AND cal.month IN (11, 12)
  AND f.cancelled = false
  AND f.diverted = false
ORDER BY (f.arr_delay + f.dep_delay) DESC, f.flight_id
```

MongoDB query

```
db.flights.find(
  { "delays.arr_delay": { $gt: 60 }, "delays.dep_delay": {
$gt: 30 }, year: 2007, month: { $in: [11, 12] }, cancelled:
false, diverted: false },
  {flight_id: 1, flight_num: 1, "delays.arr_delay": 1,
"delays.dep_delay": 1, "carrier.name": 1, "aircraft.tail_num":
1, "aircraft.total_flights": 1, "aircraft.primary_airline": 1,
"origin.city.city_name": 1, "dest.city.city_name": 1,
"origin.city.timezone_offset": 1, "dest.city.timezone_offset":
1})
.sort({ "delays.arr_delay": -1, "delays.dep_delay": -1,
flight_id: 1 })
```

Average time:
294.67 ms

Min time: 292.58 ms
Max time: 298.41 ms
Std. derivation: ± 2.01 ms

Excluding first 2 runs, using 10 executions

Average time:
49.34 ms

Min time: 45.94 ms
Max time: 53.62 ms
Std. derivation: ± 2.75 ms

Excluding first 2 runs, using 10 executions

**MongoDB
wins by 5.97x**

- Biggest win of MongoDB, every info is in the flight document.
- When accessing data that spans multiple related entities, SQL joins require complex query planning and become much less effective, even without the need for aggregation.

Self-lookup + multi-stage aggregation

Aircrafts that operated the same route multiple times on a single day, aggregating flight frequency and average delay.

TEST RESULTS

PostgreSQL query

```
SELECT a.tail_num, r.origin, r.dest, cal.year, cal.month, cal.day_of_month,
COUNT(*) as flights_per_day, AVG(f.arr_delay) as avg_delay, MIN(f.dep_time)
as first_departure, MAX(f.arr_time) as last_arrival
FROM flights f
JOIN aircraft a ON f.tail_num = a.tail_num
JOIN routes r ON f.route_id = r.route_id
JOIN calendar cal ON f.calendar_id = cal.calendar_id
WHERE f.cancelled = false AND a.total_flights > 100
GROUP BY a.tail_num, r.origin, r.dest, cal.year, cal.month, cal.day_of_month
HAVING COUNT(*) > 1
ORDER BY flights_per_day DESC, avg_delay DESC
```

MongoDB query

```
db.flights.aggregate([
{$match:{cancelled:false,"aircraft.total_flights":{$gt:100},"aircraft.tail_num":{$ne:null}}},
{$group:{_id:
{tail_num:"$aircraft.tail_num",origin:"$origin.iata",dest:"$dest.iata",year:"$year",month:"$month",day:"$day_of_month"},
flights_per_day:{$sum:1},avg_delay:{$avg:"$delays.arr_delay"},first_departure:{$min:"$dep_time"},last_arrival:
{$max:"$arr_time"}}},
{$match:{flights_per_day:{$gt:1}}},
{$sort:{flights_per_day:-1,avg_delay:-1}},
{$limit:100},
{$lookup:{from:"aircraft",localField:"_id.tail_num",foreignField:"tail_num",as:"aircraft_details"}},
{$unwind:"$aircraft_details"},
{$project:{tail_num:"$_id.tail_num",origin:"$_id.origin",dest:"$_id.dest",year:"$_id.year",month:"$_id.month",day:"$_id.day",
flights_per_day:1,avg_delay:1,first_departure:1,last_arrival:1,_id:0}},
{maxTimeMS:10000})
```

PostgreSQL
wins: MongoDB DNF

Average time: 1604.29 ms Min time: 1574.82 ms Max time: 1653.39 ms Std. derivation: ± 24.82 ms Excluding first 2 runs, using 10 executions	Average time: DNF Did not finish Query exceeded timeout limit
---	---

- PostgreSQL wins by default, since MongoDB DNF.
- MongoDB query is too complex and can't make efficient use of indexes.
- MongoDB goes over the timeout limit for every single one of the test runs.

Test name	PostgreSQL (in ms)	MongoDB (in ms)	Winner	Comparison
Mass Insertion	72.76	40.88	MongoDB	wins by 1.78x
Mass Deletion	48.02	23.64	MongoDB	wins by 2.03x
Aggregation with JOIN	433.14	1484.10	PostgreSQL	wins by 3.42x
Heavy aggregation without JOIN	180.05	883.42	PostgreSQL	wins by 4.91x
Embedded field access	583.84	215.81	MongoDB	wins by 2.71x
Range nested query	190.40	1000.62	PostgreSQL	wins by 5.26x
Embedded field increment	180.97	131.44	MongoDB	wins by 1.38x
Conditional aggregation	272.49	359.53	PostgreSQL	wins by 1.31x
Multi-table JOIN without aggregation	294.67	49.34	MongoDB	wins by 5.97x
Self-lookup + aggregation	1604.29	DNF	PostgreSQL	wins by DNF

CONCLUSIONS

SQL (PostgreSQL) advantages

- JOINS
- Aggregations
- Analytical queries
- Complex filtering
- Support for range queries
- Data integrity constraints.
- Support for algebra set operations (e.g. EXCEPT)

Shines when complex data relationships require multi-table joins with referential integrity

Document (MongoDB) advantages

- Mass write/delete operations
- Embedded field access
- Horizontal scaling
- Flexibility
- Hierarchical data

Is optimal for querying documents with self-contained, embedded sub-documents

CONCLUSIONS

To conclude this presentation, there is a clear trade-off in terms of performance:

- When there is the need for more schematic, predictable, analytical queries, applications have a clear preference for SQL.
- When there's the need for flexibility and horizontal scalability, document-based databases, such as MongoDB, offer a wide array of advantages.

The performance gap is most dramatic in complex joins and aggregations where PostgreSQL consistently outperforms MongoDB.

Conversely, MongoDB's advantages are most visible in mass operations and nested field accesses that allow for skipping joins entirely on documents.



**THANK YOU
FOR YOUR
ATTENTION**

Niccolò Buonfiglio 1983405